

Gymnázium, Praha 6, Arabská 14

Programování

Casino Web

Ročníkový projekt



Gymnázium, Praha 6, Arabská 14

Předmět: Programování

Téma: Webová aplikace s online kasinovými hrami

Školní rok: 2025/2026

Autoři: Otakar Genzer, Oliver Vachta, Josef Pižl

Třída: 3.E

Vedoucí práce: Mgr. Jan Lána

Třídní učitel: Mgr. Jan Tuzar

Prohlašujeme, že jsme jedinými autory tohoto projektu, všechny citace jsou řádně označené a všechna použitá literatura a další zdroje jsou v práci uvedené. Tímto dle zákona 121/2000 Sb. (tzv. Autorský zákon) ve znění pozdějších předpisů uděluji bezúplatně škole Gymnázium, Praha 6, Arabská 14 oprávnění k výkonu práva na rozmnožování díla (§ 13) a práva na sdělování díla veřejnosti (§ 18) na dobu časově neomezenou a bez omezení územního rozsahu

V Praze dne 14.5. 2026

Genzer

Vachta

Pižl

Anotace

Tento ročníkový projekt se zabývá návrhem a vytvořením webové aplikace simulující prostředí kasina. Cílem práce bylo naprogramovat funkční herní systém a zároveň si vyzkoušet spolupráci v týmu při vývoji webové aplikace. Aplikace obsahuje vybrané casino hry a umožňuje uživateli provádět sázky, generovat náhodné výsledky a vyhodnocovat výhry. Při realizaci byly využity webové technologie a práce byla rozdělena mezi jednotlivé členy týmu. Výsledkem je funkční webová aplikace poskytující uživateli interaktivní herní zážitek.

This annual project focuses on the design and development of a web application simulating a casino environment. The aim of the project was to create a functional gaming system and to gain experience in teamwork during web application development. The application includes selected casino games and allows the user to place bets, generate random outcomes, and evaluate winnings. Web technologies were used in the implementation, and the work was divided among the team members. The result is a functional web application that provides the user with an interactive gaming experience.

Diese Jahresarbeit beschäftigt sich mit dem Entwurf und der Entwicklung einer Webanwendung, die eine Casino-Umgebung simuliert. Ziel der Arbeit war es, ein funktionierendes Spielsystem zu programmieren und gleichzeitig Erfahrungen in der Teamarbeit bei der Entwicklung von Webanwendungen zu sammeln. Die Anwendung enthält ausgewählte Casinospiele und ermöglicht es dem Benutzer, Einsätze zu platzieren, zufällige Ergebnisse zu generieren und Gewinne auszuwerten. Bei der Umsetzung wurden Webtechnologien verwendet und die Arbeit wurde unter den Teammitgliedern aufgeteilt. Das Ergebnis ist eine funktionierende Webanwendung, die dem Benutzer ein interaktives Spielerlebnis bietet.

Obsah

Anotace.....	4
Obsah.....	5,6
1. Zadání a cíle projektu.....	7
1.1 Zadání projektu.....	7
1.2 Cíle práce.....	7
1.3 Požadavky na aplikaci.....	7
2. Architektura a použité technologie.....	8
2.1 Celková architektura systému.....	8
2.2 Backend.....	8
2.3 Frontend.....	9
2.4 Databáze.....	9
2.5 Použité nástroje.....	10
3. Implementace her a funkcí.....	11
3.1 Přehled implementovaných her.....	11
3.2 Blackjack.....	12
3.2.1 Pravidla hry.....	12
3.2.2 Implementace logiky.....	12
3.2.3 Uživatelské rozhraní.....	14
3.3 Ruleta.....	15
3.3.1 Princip hry.....	15
3.3.2 Generování výsledků.....	15
3.3.3 Sázení a vyhodnocení.....	16
Viz foto:.....	17
3.4 Sloty.....	18
3.4.1 Princip hry.....	18
3.4.2 Náhodné generování.....	18
3.4.3 Výherní kombinace.....	20
3.5.4 Vzhled slotů.....	22
3.5 Poker.....	23
3.5.1 Princip hry.....	23
3.5.2 Implementace logiky.....	23
3.5.3 Multiplayer / komunikace mezi hráči.....	23
3.5.4 Uživatelské rozhraní.....	23
3.6 Další funkce systému.....	24
3.6.1 Battle Pass.....	24
3.6.2 Questy / úkoly.....	25
3.6.3 Úprava vzhledu aplikace.....	25
3.6.4 Leaderboard.....	26

3.6.5 Správa uživatele.....	27
4. Problémy při vývoji a budoucí vize.....	28
4.1 Technické problémy.....	28
4.2 Týmová spolupráce.....	28
4.3 Vylepšení do budoucna.....	28
5. Zhodnocení a závěr.....	29
5.1 Splnění cílů.....	29
5.2 Přínos projektu.....	29
5.3 Celkové zhodnocení.....	29
Seznam zdrojů.....	30
Generace obrázků.....	31

1. Zadání a cíle projektu

1.1 Zadání projektu

V rámci tohoto ročníkového projektu jsme si stanovili za cíl navrhnout a implementovat webovou aplikaci simulující prostředí online kasina. Stránka umožňuje uživatelům vytvářet účty, spravovat virtuální herní měnu a využívat ji při hraní vybraných her, jako jsou blackjack, ruleta a online poker. Součástí systému jsou také prvky podporující aktivitu uživatelů, například battle pass a žebříček hráčů.

1.2 Cíle práce

Hlavním cílem práce bylo vytvořit funkční webovou aplikaci, která umožňuje simulaci kasinových her v online prostředí. Dílčími cíli bylo navrhnout přehledné a uživatelsky přívětivé rozhraní, implementovat herní logiku jednotlivých her, zajistit správu uživatelských účtů a virtuální měny a vyzkoušet si efektivní spolupráci v týmu.

1.3 Požadavky na aplikaci

Mezi základní funkční požadavky patří možnost registrace a přihlášení uživatele, správa virtuální měny, implementace her (blackjack, ruleta, sloty a online poker), systém sázek a vyhodnocování výher, leaderboard a doplňkové herní mechanismy, jako jsou questy a battle pass.

Mezi nefunkční požadavky patří především přehlednost a intuitivnost uživatelského rozhraní, responzivní design, dostatečný výkon aplikace a spolehlivost při práci s uživatelskými daty. Důraz byl kladen také na jednoduchou ovladatelnost a celkový uživatelský komfort.

2. Architektura a použité technologie

2.1 Celková architektura systému

Aplikace je navržena jako webový systém založený na architektuře klient-server. Uživatelské rozhraní (frontend) běží v internetovém prohlížeči, zatímco aplikační logika, zpracování dat a komunikace s databází probíhá na serveru (backend).

Komunikace mezi klientem a serverem probíhá pomocí HTTP požadavků, nejčastěji ve formátu JSON. Klient odesílá požadavky na konkrétní endpointy a server vrací odpověď, která obsahuje potřebná data. Tento princip je využíván například při hraní her nebo načítání dat battle passu a úkolů.

Například při spuštění hry sloty frontend odešle požadavek na server:

```
fetch('/api/spin/', {  
  method: 'POST',  
  body: JSON.stringify({ bet: bet })  
})
```

Server požadavek zpracuje, vygeneruje výsledek a vrátí data, která frontend zobrazí uživateli. Tento model zajišťuje oddělení prezentační a logické vrstvy aplikace.

2.2 Backend

Backend aplikace je realizován pomocí frameworku Django, který zajišťuje zpracování HTTP požadavků, práci s databází a řízení aplikační logiky.

Serverová část obsahuje jednotlivé API endpointy, například:

- /api/spin/ - generování výsledků slotů
- /api/battlepass/ - načítání dat o úrovni hráče
- /api/quests/ - načítání úkolů

Backend provádí:

- generování náhodných herních výsledků
- vyhodnocování výher
- správu uživatelských účtů a zůstatků
- ukládání a načítání dat z databáze

Díky tomu, že je hlavní logika umístěna na serveru, je zajištěna férovost hry a bezpečnost aplikace. Uživatel nemůže ovlivnit výsledek her manipulací s klientským kódem.

2.3 Frontend

Frontend aplikace je vytvořen pomocí HTML, CSS a JavaScriptu. Pro generování stránek je využíván templating systém Django, který umožňuje dynamicky vkládat data do HTML šablon.

Frontend zajišťuje:

- zobrazení herního rozhraní
- interakci s uživatelem (tlačítka, formuláře)
- animace (např. otáčení slotů)
- komunikaci se serverem pomocí funkce `fetch()`

Příklad aktualizace uživatelského rozhraní na základě dat ze serveru:

```
document.getElementById('xpBar').style.width = data.xp_progress + '%';
```

Frontend tedy reaguje na akce uživatele, odesílá požadavky na backend a následně zobrazuje získaná data.

2.4 Databáze

Databáze slouží k ukládání všech důležitých dat aplikace. Backend využívá ORM (Object-Relational Mapping) frameworku Django, který umožňuje pracovat s databází pomocí objektů místo přímých SQL dotazů.

Do databáze se ukládají například:

- uživatelské účty a přihlašovací údaje
- aktuální zůstatek herní měny
- postup v Battlepassu (level, XP)
- splněné úkoly
- historie her (dle implementace)

Díky databázi je možné uchovávat stav aplikace a zajistit, že data zůstanou zachována i po odhlášení uživatele.

2.5 Použité nástroje

Při vývoji aplikace byly použity následující technologie a nástroje:

- Django - backend framework pro tvorbu webové aplikace
- HTML, CSS, JavaScript - technologie pro frontend
- GitHub - správa verzí a týmová spolupráce
- IDE - Visual Studio Code (VS code)

Tyto nástroje umožnily efektivní vývoj aplikace, správu kódu a spolupráci mezi členy týmu.

3. Implementace her a funkcí

3.1 Přehled implementovaných her

V rámci aplikace byly implementovány základní kasinové hry, konkrétně blackjack, ruleta, sloty a online poker. Tyto hry umožňují uživateli sázet virtuální herní měnu a získávat výhry na základě herní logiky a náhodných mechanismů.

Blackjack a poker jsou karetní hry, přičemž poker probíhá v online režimu mezi více hráči. Ruleta je založena na náhodném výběru čísla a různých typech sázek. Sloty fungují na principu generování náhodných symbolů a vyhodnocování výherních kombinací.

Všechny hry jsou propojeny jednotným systémem sázek a správy virtuální měny.

3.2 Blackjack

3.2.1 Pravidla hry

Hráč hraje proti dealerovi. Cílem hráče i dealera je dosáhnout hodnoty karet co nejbližší hodnotě 21, aniž by došlo k jejímu překročení, následně kdo je 21 hodnotně blíže vyhrává. Hráč když vyhraje získává dvojnásobek svého vkladu, dealer dostane sázky jestliže vyhraje. Každá karta má svou hodnotu, přičemž eso může mít hodnotu 1 nebo 11 podle aktuální situace a jasek, královna a král mají hodnotu 10.

3.2.2 Implementace logiky

Implementace blackjacku zahrnuje několik kroků. Nejprve dojde k rozdání karet hráči a dealerovi. Následně má hráč možnost rozhodnout se, zda chce další kartu nebo zůstane na aktuální hodnotě. Po dokončení tahu hráče přichází na řadu dealer, který se řídí předem definovanými pravidly. Nakonec dojde k vyhodnocení výsledku a případnému připsání výhry.

```
function createDeck() {  
  const suits = ['♠', '♥', '♦', '♣'];  
  const values = ['A', '2', '3', '4', '5', '6', '7', '8', '9', '10', 'J', 'Q', 'K'];  
  let newDeck = [];  
  
  for (let suit of suits) {  
    for (let value of values) {  
      newDeck.push({suit, value});  
    }  
  }  
  
  return newDeck.sort(() => Math.random() - 0.5);  
}
```

```
function getValue(cards, hideFirst = false) {  
  let value = 0;  
  let aces = 0;  
  let start = hideFirst ? 1 : 0;  
  
  for (let i = start; i < cards.length; i++) {  
    let card = cards[i];  
  
    if (card.value === 'A') {  
      value += 11;  
      aces++;  
    }  
  }  
}
```

```

    } else if (!['K', 'Q', 'J'].includes(card.value)) {
        value += 10;
    } else {
        value += parseInt(card.value);
    }
}

while (value > 21 && aces) {
    value -= 10;
    aces--;
}

return value;
}

function isBlackjack(cards) {
    return cards.length === 2 && getValue(cards) === 21;
}

```

3.2.3 Uživatelské rozhraní

Uživatelské rozhraní blackjacku umožňuje zobrazit karty hráče i dealera a nabízí tlačítka pro jednotlivé akce. Uživatel má přehled o aktuálním stavu hry a může snadno provádět své tahy.



3.3 Ruleta

3.3.1 Princip hry

Ruleta je hra založená na náhodném výběru čísla v rozsahu od nuly do třiceti šesti. Uživatel může sázet na různé kombinace, například konkrétní číslo, barvu nebo skupinu čísel.

3.3.2 Generování výsledků

Výsledek ruletového kola je generován pomocí náhodného algoritmu. Tento algoritmus vybírá číslo z definovaného rozsahu, přičemž každé číslo má stejnou pravděpodobnost výběru.

Generování výsledku rulety probíhá na serverové straně, kam jsou odeslány všechny aktuální sázky. Funkce `spinRoulette()` shromáždí sázky od uživatele a odešle je pomocí HTTP požadavku na backend. Server následně vygeneruje náhodné číslo a vrátí ho spolu s dalšími informacemi zpět klientovi. Funkce `getNumberColor()` slouží k určení barvy vylosovaného čísla na základě předem definované množiny červených čísel.

```
const redNumbers = new Set([1, 3, 5, 7, 9, 12, 14, 16, 18, 19, 21, 23, 25, 27, 30, 32, 34, 36]);
const wheelOrder = [0, 32, 15, 19, 4, 21, 2, 25, 17, 34, 6, 27, 13, 36, 11, 30, 8, 23, 10, 5, 24, 16, 33, 1, 20, 14, 31, 9, 22, 18, 29, 7, 28];
```

```
function getNumberColor(number) {
  if (number === 0) return 'green';
  return redNumbers.has(number) ? 'red' : 'black';
}
```

```
function spinRoulette() {
  if (spinning) return;

  const payloadBets = Array.from(bets.values()).map(bet => ({
    type: bet.type,
    value: bet.value,
    amount: bet.amount
  }));

  fetch('/api/spin-roulette/', {
    method: 'POST',
    headers: {
      'Content-Type': 'application/json',

      body: JSON.stringify({ bets: payloadBets })
    }
  });
}
```

```

    })
    .then(response => response.json())
    .then(data => {
        showResult(data);
    });
}

```

3.3.3 Sázení a vyhodnocení

Po uzavření sázek dojde k vygenerování výsledku. Systém následně vyhodnotí všechny sázky a vypočítá výhry podle typu sázky a platných pravidel. Výsledky jsou následně zobrazeny uživateli a kredit je upraven.

Sázení je realizováno funkcí `addBet()`, která ukládá jednotlivé sázky do datové struktury typu `mapa`. Pokud uživatel vsadí na stejné pole vícekrát, částka se automaticky navyšuje. Po vygenerování výsledku server vrátí informace o výhře nebo prohře, které jsou zpracovány funkcí `showResult()`. Tato funkce aktualizuje zůstatek hráče, zobrazí výsledné číslo včetně jeho barvy a informuje uživatele o výsledku hry.

```

function addBet(bet) {

    const key = `${bet.type}:${bet.value}`;

    const current = bets.get(key);

    if (current) {

        current.amount += selectedChip;
        current.lastChip = selectedChip;
    } else {
        bets.set(key, { ...bet, amount: selectedChip, lastChip: selectedChip });
    }
    renderBets();
}

function showResult(data) {
    const netWin = Number(data.net_win);
    const color = data.color || getNumberColor(data.number);
    balance.textContent = data.new_balance;
    resultValue.innerHTML = `
        <span class="result-number ${color}">${data.number}</span>
        <span>${netWin > 0 ? '+' : ''}${formatMoney(netWin)}</span>
    `;
}

```



```

if (netWin > 0) {
    messageBar.textContent = `Vyhra $$${formatMoney(netWin)}!`;
} else if (Number(data.total_return) > 0) {
    messageBar.textContent = 'Cast sazký se vratila.';
} else {
    messageBar.textContent = 'Tentokrat bez vyhry';
}

```

Viz foto:



3.4 Sloty

3.4.1 Princip hry

Sloty jsou hazardní hra založená na náhodném generování symbolů na herních válcích. Hráč si nejprve zvolí výši sázky, která je následně odečtena z jeho celkového počtu peněz, a poté spustí otočení válců pomocí tlačítka „SPIN“.

V implementaci jsou jednotlivé válce reprezentovány jako grafické prvky obsahující sadu symbolů, které se vizuálně posouvají pomocí animace. Spuštění hry je realizováno funkcí `spin()`, která odešle požadavek na server:

```
fetch('/api/spin/', {  
  
  method: 'POST',  
  
  body: JSON.stringify({ bet: bet })  
  
})
```

Server následně vygeneruje výsledek hry a vrátí kombinaci symbolů, která je zobrazena uživateli. Výsledné symboly jsou vykresleny posunutím jednotlivých válců na odpovídající pozici:

```
track.style.transform = `translateY(-${symbolIndex(reelResults[index]) * 100}%)`;
```

Cílem hry je získat výherní kombinaci symbolů. Pokud se podaří vytvořit odpovídající kombinaci, hráč získává výhru, jinak o svou sázku přichází. Celý proces je řízen serverem, zatímco klientská část zajišťuje pouze vizualizaci a interakci s uživatelem.

3.4.2 Náhodné generování

Princip fungování slotů je založen na komunikaci mezi frontendem a backendem. Po stisknutí tlačítka „SPIN“ dojde k odeslání požadavku na server, který vygeneruje náhodný výsledek hry.

Tento proces je realizován funkcí spin():

```
function spin() {
  if (isSpinning) return;

  const bet = getBetValue();

  if (!bet || bet <= 0) {
    alert('Please enter a valid bet amount!');
    return;
  }

  const balance = parseFloat(document.getElementById('balance').textContent);
  if (balance < bet) {
    alert('Insufficient balance!');
    return;
  }

  isSpinning = true;

  fetch('/api/spin/', {
    method: 'POST',
    headers: {
      'Content-Type': 'application/json',
    },
    body: JSON.stringify({ bet: bet })
  })
  .then(response => response.json())
  .then(data => {
    setTimeout(() => {
      displayResults(data);
    }, 800);
  });
}
```

Tato funkce nejprve zkontroluje platnost sázky a dostatečný zůstatek hráče. Následně odešle požadavek na server (/api/spin/), který vygeneruje náhodný výsledek hry a vrátí jej zpět ve formátu JSON.

Po obdržení odpovědi je výsledek zobrazen pomocí funkce displayResults():

```
function displayResults(data) {
  const reels = [
    document.getElementById('reel1'),
```

```

        document.getElementById('reel2'),
        document.getElementById('reel3')
    ];

    const reelResults = data.reels;

    reels.forEach((reel, index) => {
        const track = reel.querySelector('.reel-track');
        track.style.transform =
            `translateY(-${symbolIndex(reelResults[index]) * 100}%)`;
    });

    document.getElementById('balance').textContent = data.new_balance;
}

```

Tato funkce nastaví výsledné symboly na válcích podle dat ze serveru a aktualizuje zůstatek hráče.

Celý proces tak funguje následovně:

1. hráč stiskne tlačítko „SPIN“
2. frontend odešle požadavek na backend
3. backend vygeneruje náhodný výsledek
4. frontend zobrazí výsledek a aktualizuje stav hry

Tímto způsobem je zajištěno, že náhodné generování probíhá na serveru, což zvyšuje férovost a bezpečnost aplikace.

3.4.3 Výherní kombinace

Výherní kombinace ve hře sloty jsou určeny podle výsledné kombinace symbolů na jednotlivých válcích. Po každém otočení jsou symboly vyhodnoceny a podle jejich shody je určena výše výhry.

V základní implementaci platí, že pokud se na všech třech válcích objeví stejné symboly, hráč získává násobek své sázky. Výše násobku závisí na konkrétním symbolu (např. diamanty mají vyšší hodnotu než běžné symboly).

Dále je implementována i částečná výhra - pokud se shodují alespoň dva symboly, hráč získává menší výhru, konkrétně 1,5násobek sázky.

Tato logika je v klientské části realizována například ve funkci `calculateDisplayWin()`:

```
const counts = {};  
  
normalizedReels.forEach(symbol => {  
  counts[symbol] = (counts[symbol] || 0) + 1;  
});  
  
const maxMatch = Math.max(...Object.values(counts));  
  
if (maxMatch === 2) {  
  return bet * 1.5;  
}
```

Na základě počtu stejných symbolů je tedy určeno, zda hráč vyhrává, a případně jak velká bude jeho výhra. Kompletní vyhodnocení je následně kombinováno s výsledkem ze serveru, který určuje konečnou hodnotu výhry.

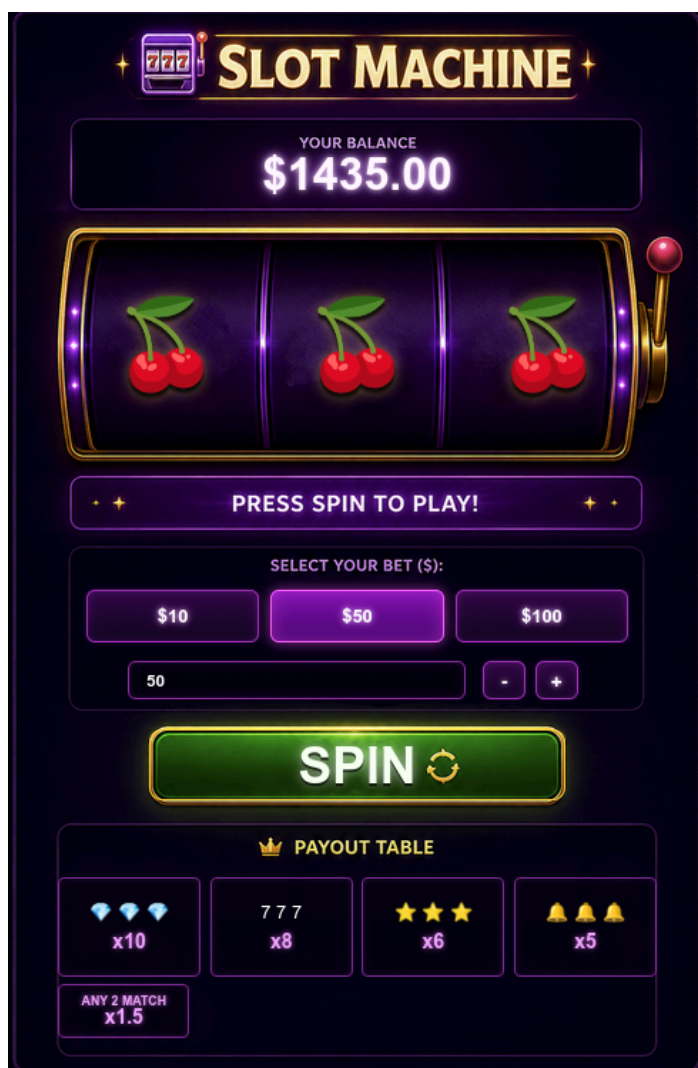
3.5.4 Vzhled slotů

Uživatelské rozhraní hry sloty je navrženo tak, aby vizuálně připomínalo moderní online kasino. Hlavním prvkem je samotný slotový automat se třemi válci, na kterých se zobrazují symboly.

Nad válci je zobrazen aktuální zůstatek hráče, pod nimi se nachází ovládací prvky pro nastavení sázky a tlačítko „SPIN“.

Součástí rozhraní je také tabulka výher (payout table), která informuje hráče o možných kombinacích a jejich násobcích. Celé rozhraní je doplněno animacemi, které se aktivují při otáčení válců a při výhře, čímž se zvyšuje herní zážitek uživatele.

Obrázek slotů:



3.5 Poker

3.5.1 Princip hry

Hráči mohou založit lobby, do kterého se může připojit jeden další hráč. Po té, co jsou oba připravení hrají Poker podle klasických pravidel Texas Hold'em. Všechny sázky se odečítají z jejich počátečního vkladu a po dohrání ruky si hráči mohou peníze vybrat.

3.5.2 Implementace logiky

Poker je implementovaný jako serverově řízená hra v Django aplikaci. Herní stav je uložen v databázi pomocí modelů pro hru, hráče, jednotlivé handy, karty hráčů a provedené akce. Každá pokerová hra obsahuje seznam aktivních hráčů, aktuální handu, blindy, pot a pořadí hráčů.

Při vytvoření hry se nastaví minimální a maximální buy-in, small blind a big blind. Po připojení dvou hráčů se vytvoří nová handa ve stavu čekání. Hráči musí potvrdit připravenost, poté server rozdá karty, nastaví blindy, naplní počáteční pot a určí hráče na tahu.

Veškeré akce jako fold, call/check a bet/raise se odesílají na server. Server ověřuje, zda je hráč skutečně na tahu, upravuje stack hráče, pot, aktuální sázku a zaznamenává akce do historie. Po dokončení sázkového kola se hra automaticky posouvá mezi fázemi pre-flop, flop, turn a river. Po skončení handy server určí vítěze, připíše mu pot a připraví další handu.

3.5.3 Multiplayer / komunikace mezi hráči

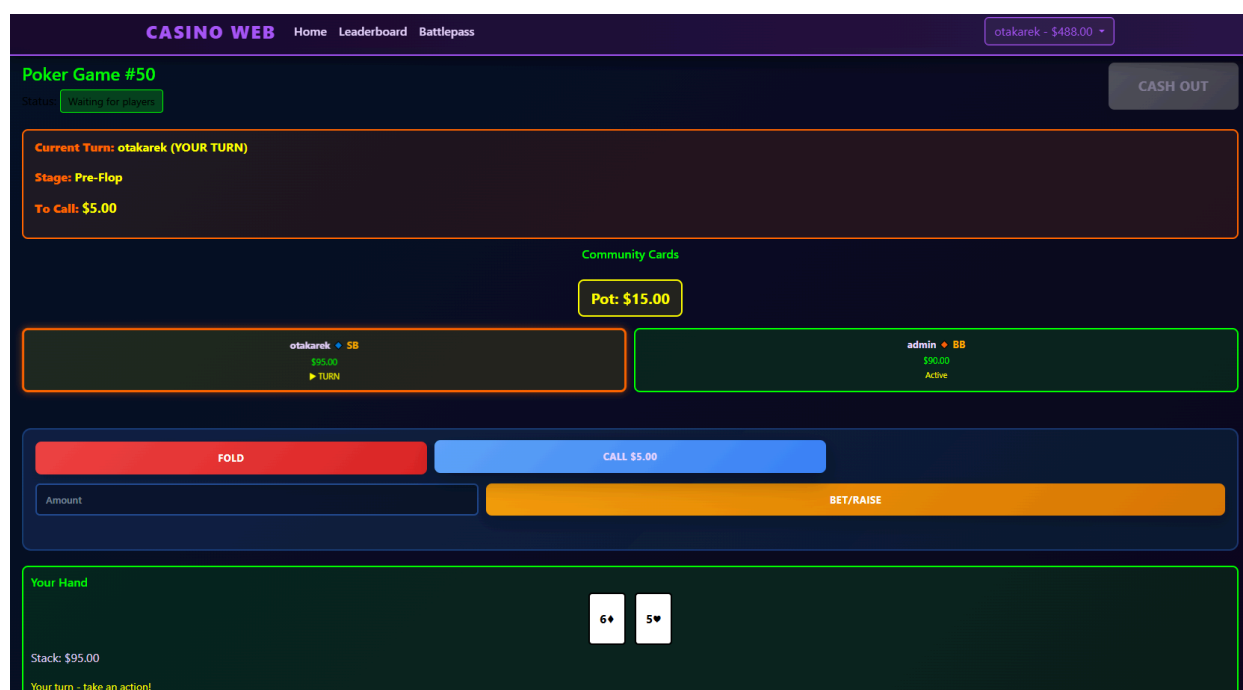
Multiplayer funguje přes sdílený stav uložený na serveru. Každý hráč je veden jako samostatný záznam v databázi a je přiřazen ke konkrétní pokerové hře.

Klientská část pravidelně posílá požadavek na aktuální stav hry. Server vrací informace o hráčích, jejich stacku, aktuálním tahu, fázi hry, komunitních kartách, potu a případném vítězi poslední handy. Tím se rozhraní obou hráčů průběžně synchronizuje bez nutnosti ručního obnovování stránky.

Akce hráče jsou posílány pomocí POST požadavků. Server vždy kontroluje, zda má daný hráč právo akci provést. Pokud není na tahu, akce se odmítne. Tím se zabraňuje tomu, aby hráč zahrál mimo pořadí nebo ovlivnil hru neoprávněně.

3.5.4 Uživatelské rozhraní

Uživatelské rozhraní pokeru je vytvořené jako samostatná herní stránka. Zobrazuje základní informace o hře, aktuální fázi handy, hráče na tahu, částku potřebnou k dorovnání, komunitní karty, pot, hráče u stolu a vlastní karty přihlášeného hráče. Ovládací prvky se zobrazují pouze hráči, který je právě na tahu.



3.6 Další funkce systému

Kromě samotných her obsahuje aplikace také doplňkové funkce, které rozšiřují herní zážitek a zvyšují interaktivitu systému. Mezi tyto funkce patří battle pass, systém úkolů, úprava vzhledu aplikace a leaderboard.

3.6.1 Battle Pass

Battle pass představuje systém postupného odměňování hráče na základě jeho aktivity. Uživatel získává zkušenosti (XP) především hraním jednotlivých her a plněním úkolů. Na základě získaných zkušeností se zvyšuje jeho úroveň, což odemýká nové odměny.

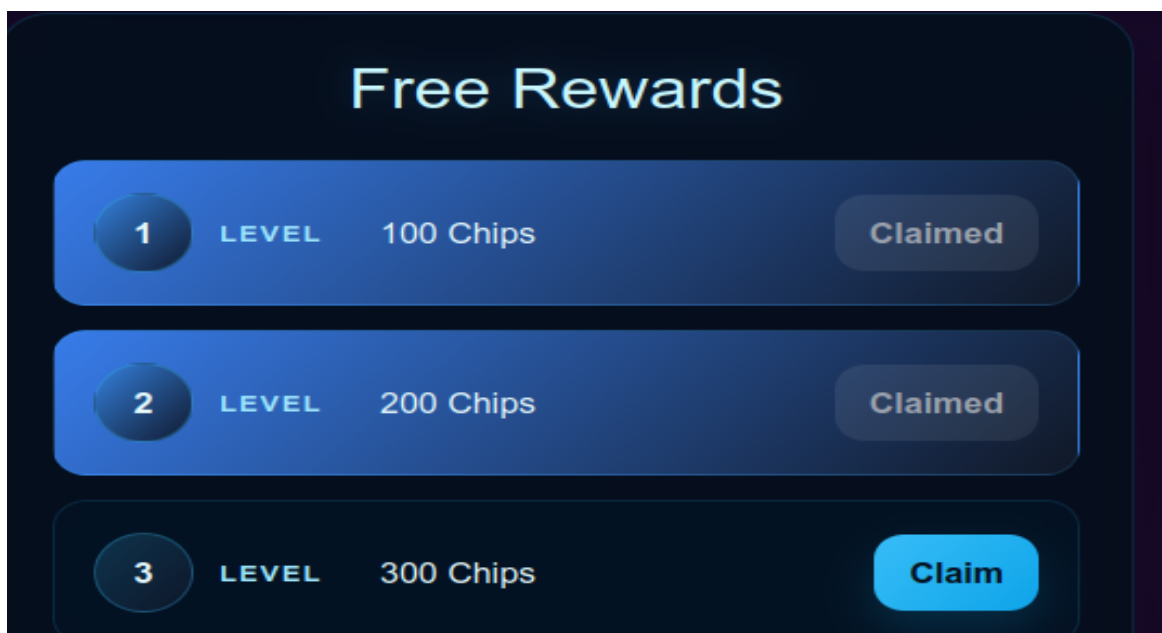
Data battle passu jsou dynamicky načítána z backendu pomocí API endpointu:

```
fetch('/api/battlepass/')
```

Po obdržení dat dochází k aktualizaci uživatelského rozhraní, například zobrazení aktuální úrovně a postupu do další úrovně:

```
document.getElementById('levelBadge').textContent = data.level;  
document.getElementById('xpBar').style.width = data.xp_progress + '%';
```

Součástí systému je také mechanismus odměn, které si hráč může vyzvednout po dosažení určité úrovně. Každá úroveň má přiřazenou konkrétní odměnu. Tento systém motivuje hráče k pravidelnému hraní a postupnému zlepšování.



3.6.2 Questy / úkoly

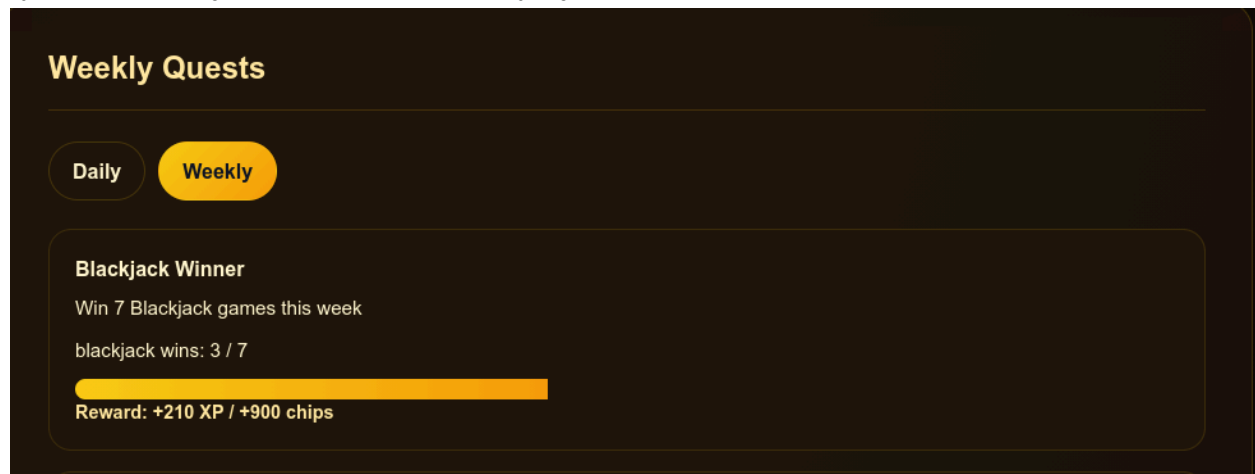
Systém úkolů rozšiřuje herní mechaniky o konkrétní cíle, které může hráč plnit. Úkoly jsou rozděleny například na denní a týdenní a jsou pravidelně obnovovány.

Data úkolů jsou načítána z backendu pomocí API:

```
fetch('/api/quests/')
```

Po načtení jsou úkoly dynamicky vykresleny na stránce a zobrazují aktuální postup hráče, například kolik her již odehrál nebo kolik výher získal. Součástí zobrazení je také ukazatel postupu (progress bar), který vizuálně znázorňuje plnění úkolu.

Splněním úkolů hráč získává odměny, například zkušenosti (XP) nebo herní měnu. Tento systém podporuje pravidelné hraní a zvyšuje variabilitu herního prostředí.



3.6.3 Úprava vzhledu aplikace

Aplikace umožňuje uživateli přizpůsobit si vzhled pomocí výběru různých motivů. Tato funkce umožňuje personalizaci prostředí.

Výběr motivu probíhá prostřednictvím formuláře s výběrem možností:

```
<input type="radio" name="theme" value="{{ value }}">
```

Zvolený motiv je uložen do uživatelského profilu a následně ovlivňuje vzhled celé aplikace, například barvy pozadí, tlačítek nebo herních prvků. Díky tomu si může každý uživatel přizpůsobit aplikaci podle svých preferencí.

Nastavení vzhledu

Vyber si motiv pro web a hru. Volbu uložíme do tvého profilu.

☐ Neon

☒ Gold

☐ Retro Vegas

☐ Minimal

Uložit motiv

3.6.4 Leaderboard

Leaderboard slouží k porovnání hráčů mezi sebou na základě jejich herního zůstatku. Uživatelé jsou seřazeni podle výše svého kreditu a jejich pořadí je zobrazeno ve formě žebříčku.

Každý záznam obsahuje pořadí hráče, jeho uživatelské jméno a aktuální zůstatek:

```
<div class="leaderboard-rank">{{ forloop.counter }}</div>
<div class="leaderboard-player">{{ profile.user.username }}</div>
<div class="leaderboard-balance">${{ profile.balance }}</div>
```

Nejlepší hráči jsou vizuálně zvýrazněni (například zlatá, stříbrná a bronzová pozice), což zvyšuje přehlednost a atraktivitu žebříčku. Tento systém podporuje soutěživost mezi uživateli a motivuje je k lepším výkonům ve hrách.

Leaderboard		
All players ranked by current balance		
7 players		
1	oliver.vachta.s@gyarab.cz	\$2020.00
2	pesek3	\$930.00

3.6.5 Správa uživatele

Aplikace obsahuje systém správy uživatelů, který zahrnuje registraci, přihlášení a práci s uživatelským účtem. Přihlášení probíhá pomocí formuláře, kde uživatel zadává své přihlašovací údaje:

```
<input type="email" name="login">  
<input type="password" name="password">
```

Po přihlášení má uživatel přístup ke svému účtu, kde může sledovat svůj herní zůstatek a využívat další funkce aplikace. Součástí systému je také jednoduchý obchod, který umožňuje doplnění herní měny:

```
<button type="submit" name="amount" value="100">Buy 100</button>
```

Tento systém zajišťuje ukládání uživatelských dat, personalizaci prostředí a celkovou funkčnost aplikace z pohledu uživatele.

4. Problémy při vývoji a budoucí vize

4.1 Technické problémy

Během vývoje aplikace jsme narazili na několik technických problémů, zejména při komunikaci mezi frontendem a backendem. Například správné odesílání dat pomocí fetch API a práce s CSRF tokenem v Django vyžadovala důkladné pochopení principu HTTP požadavků.

Další problém představovalo náhodné generování výsledků her a jejich správné zobrazení na frontendové části. Bylo nutné zajistit, aby výsledky byly generovány na serveru a následně správně synchronizovány s animacemi na straně klienta.

V některých složitějších situacích jsme využili dostupné online zdroje a dokumentaci.

4.2 Týmová spolupráce

Projekt byl realizován v týmu, což přineslo jak výhody, tak i určité komplikace. Mezi hlavní problémy patřilo rozdělení práce a time management. V některých fázích vývoje nebyly úkoly rozděleny zcela rovnoměrně, což vedlo k nerovnoměrnému vytížení jednotlivých členů týmu.

Postupem času se nám však podařilo lépe organizovat práci, komunikovat mezi sebou a efektivněji si rozdělovat jednotlivé části projektu (např. frontend, backend, jednotlivé hry).

Motivace v průběhu projektu kolísala, zejména při řešení složitějších problémů, nicméně díky postupnému pokroku a viditelným výsledkům se nám podařilo projekt dokončit.

4.3 Vylepšení do budoucna

Do budoucna by bylo možné aplikaci dále rozšířit o další funkce a vylepšení. Například:

- přidání více her a herních režimů
- vylepšení grafického rozhraní a animací
- implementace multiplayer funkcí ve více hrách
- rozšíření systému odměn (battle pass, questy)
- lepší optimalizace aplikace a zabezpečení
- zpřístupnit poker pro více než 2 hráče v jednom lobby

Dále by bylo možné aplikaci nasadit na veřejný server a zpřístupnit ji širšímu okruhu uživatelů.

5. Zhodnocení a závěr

5.1 Splnění cílů

Hlavním cílem projektu bylo vytvořit funkční webovou aplikaci s kasinovými hrami. Tento cíl se podařilo splnit, jelikož aplikace obsahuje několik her (blackjack, ruleta, sloty, poker) a doplňkové funkce jako battle pass, questy a leaderboard.

5.2 Přínos projektu

Projekt přinesl především praktické zkušenosti s vývojem webových aplikací. Naučili jsme se pracovat s frameworkem Django, komunikací mezi frontendem a backendem, správou dat a týmovou spoluprací.

Zároveň jsme si osvojili principy návrhu aplikací a řešení reálných problémů, které se při vývoji vyskytují.

5.3 Celkové zhodnocení

Celkově lze projekt hodnotit jako úspěšný. Přestože jsme se během vývoje setkali s různými problémy, podařilo se nám je postupně vyřešit a vytvořit funkční aplikaci. Projekt nám poskytl cenné zkušenosti a ukázal, jak probíhá vývoj složitějšího softwarového systému v praxi.

Seznam zdrojů

DJANGO SOFTWARE FOUNDATION. Writing your first Django app [online]. Dostupné z: <https://docs.djangoproject.com/en/stable/intro/tutorial01/> [cit. 2026-05-04].

MDN WEB DOCS. Fetch API [online]. Dostupné z: https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API [cit. 2026-05-04].

MDN WEB DOCS. Working with JSON [online]. Dostupné z: <https://developer.mozilla.org/en-US/docs/Learn/JavaScript/Objects/JSON> [cit. 2026-05-04].

STACK OVERFLOW. How to shuffle an array in JavaScript? [online]. Dostupné z: <https://stackoverflow.com/questions/2450954/how-to-randomize-shuffle-a-javascript-array> [cit. 2026-05-04].

STACK OVERFLOW. How to send POST request using fetch API? [online]. Dostupné z: <https://stackoverflow.com/questions/29775797/fetch-post-json-data> [cit. 2026-05-04].

STACK OVERFLOW. CSRF token missing or incorrect in Django [online]. Dostupné z: <https://stackoverflow.com/questions/17716624/django-csrf-token-missing-or-incorrect> [cit. 2026-05-04].

YOUTUBE. Web Dev Simplified. JavaScript Fetch API Tutorial - Full Course [online]. Dostupné z: <https://www.youtube.com/watch?v=cuEtnrL9-H0> [cit. 2026-05-04].

YOUTUBE. Code Explained. Blackjack Game in JavaScript [online]. Dostupné z: <https://www.youtube.com/watch?v=bMYCWccL-3U> [cit. 2026-05-04].

YOUTUBE. CodingNepal. Slot Machine in JavaScript [online]. Dostupné z: <https://www.youtube.com/watch?v=E3XxeE7NF30> [cit. 2026-05-04].

YOUTUBE. The Coding Train. Roulette Simulation in JavaScript [online]. Dostupné z: <https://www.youtube.com/watch?v=9Y7d5K2k2bY> [cit. 2026-05-04].

GITHUB. gyarab. 2025-3e-casino_web [online]. Dostupné z: https://github.com/gyarab/2025-3e-casino_web [cit. 2026-05-04].

Generace obrázků

CHAT GPT. "Vygeneruj mi tapetu pro blackjack, pouze stůl s žetonama a balíčkem karet."

Dostupné z: <https://chatgpt.com/c/69f50330-a614-838b-92db-381a718bfaf3> [cit. 2026-05-04].

CHAT GPT. "udělej mi hezkou slot machine pro moje casino." Dostupné z:

<https://chatgpt.com/c/69f50330-a614-838b-92db-381a718bfaf3> [cit. 2026-05-04].

CHAT GPT. "Udělej mi hrací plochu pro online roulette ve stylu designu slot machine." Dostupné

z: <https://chatgpt.com/c/69f50330-a614-838b-92db-381a718bfaf3> [cit. 2026-05-04].